

Prototipo de IDS con IA Explicable (XAI) para mejorar la detección de intrusiones en las redes SOC mediante el Dataset NSL-KDD, 2026.

IDS prototype with Explainable AI (XAI) to improve intrusion detection in SOC networks using the NSL-KDD Dataset, 2026.

A. Principe^{1,*,*}, C. Santana^{2,*}, and Y. Tello^{3,*}

¹Escuela profesional de Ciberseguridad, Universidad Nacional de Ingeniería, Av. Túpac Amaru 210
Rímac, Lima, 15333, Perú

²Escuela profesional de Ciberseguridad, Universidad Nacional de Ingeniería, Av. Túpac Amaru 210
Rímac, Lima, 15333, Perú

* Autor correspondiente: A. Principe (anghelo.principe.o@uni.pe)

Abstract

Intrusion Detection Systems (IDS) based on machine learning face the technical problem of the "black box", which results in operational distrust and alert fatigue in analysts in Security Operations Centers (SOC). To mitigate this gap, an IDS prototype was developed that integrates the LightGBM classifier with the TreeSHAP explainer, optimized through a dynamic asymmetric penalty (increasing the weight of minority classes by 2.5x) to reduce false positives in privilege escalations. Unlike previous studies focused on laboratory environments, this system was rigorously validated against the KDDTest+ adversarial subset, which includes 17 types of "Zero-Day" attacks. The architecture incorporates a natural language translation layer that maps network variables to MITER ATT&CK framework tactics and a forensic fault diagnosis module. The results show a global accuracy (Accuracy) of 74.43% and a macro F1-score of 0.4781, highlighting a precision of 40% in the User to Root category, significantly exceeding the 1% of the unadjusted models. The training time (29.21 s) and inference latency (0.0823 ms/packet) confirm its operational viability. It is concluded that, in the face of unknown signatures where supervised learning presents detection limitations (Recall), post-hoc explainability is essential to make the logic of the model transparent and identify the causes of failed alerts in the SOC.

Keywords: IDS, SOC, XAI, NSL-KDD, SHAP, LightGBM.

Resumen

Los Sistemas de Detección de Intrusiones (IDS) basados en aprendizaje automático enfrentan el problema técnico de la "caja negra", lo cual deriva en desconfianza operativa y fatiga por alertas en los analistas de los Centros de Operaciones de Seguridad (SOC). Para mitigar este vacío, se desarrolló un prototipo de IDS que integra el clasificador LightGBM con el explicador TreeSHAP, optimizado mediante una penalización asimétrica dinámica (incrementando 2.5x el peso de las clases minoritarias) para reducir falsos positivos en escaladas de privilegios. A diferencia de estudios previos centrados en entornos de laboratorio, este sistema fue validado rigurosamente frente al subconjunto adversarial KDDTest+, que incluye 17 tipos de ataques de "Día Cero". La arquitectura incorpora una capa de traducción en lenguaje natural que mapea las variables de red hacia tácticas del marco MITRE ATT&CK y un módulo de diagnóstico forense de fallos. Los resultados muestran una exactitud global (Accuracy) de 74.43% y un F1-score macro de 0.4781, destacando una precisión del 40% en la categoría User to Root, superando significativamente el 1% de los modelos no ajustados. El tiempo de entrenamiento (29.21 s) y la latencia de inferencia (0.0823 ms/paquete) confirman su viabilidad operativa. Se concluye que, ante firmas desconocidas donde el aprendizaje supervisado presenta limitaciones de detección (Recall), la explicabilidad post-hoc resulta indispensable para transparentar la lógica del modelo e identificar las causas de las alertas fallidas en el SOC.

Palabras clave: IDS, SOC, XAI, NSL-KDD, SHAP, LightGBM

1 Introducción

La evolución de las infraestructuras de red ha expandido significativamente la superficie de ataque, derivando en amenazas de ciberseguridad cada vez más complejas y sofisticadas [1]. Ante la insuficiencia de los sistemas tradicionales basados en reglas para detectar vectores de ataque emergentes, los Centros de Operaciones de Seguridad (SOC) han adoptado Sistemas de Detección de Intrusiones (IDS) basados en aprendizaje automático y profundo [1] [2]. Sin em-

basados [1]. Ante la insuficiencia de los sistemas tradicionales basados en reglas para detectar vectores de ataque emergentes, los Centros de Operaciones de Seguridad (SOC) han adoptado Sistemas de Detección de Intrusiones (IDS) basados en aprendizaje automático y profundo [1] [2]. Sin em-

bargo, la implementación de estos modelos avanzados ha introducido el problema técnico de la "caja negra", donde la opacidad de los procesos de toma de decisiones impide que los analistas comprendan la lógica subyacente a las alertas [3] [4]. Esta carencia de transparencia genera 'alertas ciegas' y una profunda desconfianza operativa. A esta problemática se suma la incapacidad intrínseca de los modelos supervisados para reconocer patrones de ataque no documentados (Zero-Day), lo que confina su efectividad a una 'burbuja' estadística que no refleja las condiciones reales de una red SOC. Esta situación, sumada a la alta tasa de falsos positivos, deriva en una fatiga por alertas que ralentiza drásticamente la respuesta ante incidentes críticos [3].

En la revisión del estado del arte, el dataset NSL-KDD se ha consolidado como el estándar de evaluación para mitigar la redundancia estadística del KDD99 original [5]. Diversas investigaciones han implementado modelos de optimización metaheurística, como el Golden Jackal Driven Optimization (GJO) y el Harris Hawks Optimization (HHO), así como ensambles de alto rendimiento como XGBoost, alcanzando precisiones en entornos de laboratorio superiores al 99% [6] [7] [1]. No obstante, se critica que estas soluciones ensanchan la 'brecha semántica' al priorizar exclusivamente la maximización de métricas estadísticas obtenidas mediante particiones internas de datos, lo que a menudo enmascara un sobreajuste que impide la detección de amenazas externas [6]. Al centrarse en eficacias matemáticas ideales, estos modelos omiten la justificación lógica exigida por las normativas de auditoría, dificultando que el analista traduzca un peso numérico en una táctica ofensiva concreta [8].

Para abordar este vacío, se ha introducido la Inteligencia Artificial Explicable (XAI) mediante algoritmos como SHAP (SHapley Additive exPlanations), los cuales permiten auditar las variables y cuantificar su contribución a la predicción [3] [8]. A pesar de estas ventajas, se identifica una brecha científica crítica: la alta latencia computacional de los explicadores tradicionales cuando se enfrentan a volúmenes masivos de tráfico en tiempo real [9] [8]. Esta limitación es particularmente grave ante ataques silenciosos y de bajo volumen, como los de las categorías Remote to Local (R2L) y User to Root (U2R), donde la pérdida de información durante la transferencia de conocimiento o la lentitud en la generación de explicaciones impide una detección proactiva [10] [5]. Por tanto, los modelos actuales fallan en equilibrar la transparencia exigida con la agilidad necesaria para entornos de alta velocidad [9] [11].

En la presente investigación se propone el desarrollo de un prototipo optimizado que integra LightGBM con el explicador TreeSHAP, diseñado para ofrecer una atribución de importancia de características rápida y matemáticamente consistente [3] [9]. A diferencia de los enfoques convencionales, el sistema se somete a una validación externa rigurosa frente al subconjunto adversarial KDDTest+, enfrentándolo a 17 tipos de ataques de 'Día Cero' jamás observados en el entrenamiento. Esta decisión metodológica permite alcanzar una 'exactitud honesta' que, aunque numéricamente inferior a los resultados de laboratorio, ofrece una transparencia y capacidad diagnóstica superior ante la realidad operativa del SOC. A través de este diseño, se vinculan las variables independientes de eficiencia y cobertura táctica con las dependientes de efectividad F1-score y agilidad, demostrando cómo

la explicabilidad post-hoc puede diagnosticar incluso los fallos heurísticos del propio modelo. [5] [12].

2 METODOLOGÍA

El presente capítulo describe el diseño metodológico adoptado para el desarrollo y la evaluación del prototipo de detección de intrusiones dotado de un componente de inteligencia artificial explicable. Se exponen, de manera sucesiva, el tipo y diseño de investigación, la operacionalización de las variables de estudio, la población y la muestra empleadas, las técnicas e instrumentos de recolección de datos, el procedimiento de depuración y selección de características, el modelado predictivo, el marco de explicabilidad y el método de análisis de los datos. El detalle técnico-operativo de la arquitectura del sistema, de las transformaciones aplicadas y de la configuración del clasificador se desarrolla en la sección 3.

2.1 Tipo y diseño de investigación

La presente investigación se clasifica, según su finalidad, como una investigación de tipo aplicada o tecnológica, en la medida en que su propósito central consiste en el desarrollo y la evaluación empírica de un prototipo funcional orientado a resolver un problema concreto de la práctica profesional en el dominio de la ciberseguridad, antes que en la producción de conocimiento teórico de carácter general [6] [8] [13].

Según su enfoque, corresponde a una investigación de naturaleza cuantitativa, dado que la verificación del efecto del prototipo sobre la detección de intrusiones se sustenta en la medición numérica de indicadores de desempeño y de explicabilidad, los cuales son sometidos a un tratamiento estadístico de carácter descriptivo [1] [5].

En relación con su diseño, se adopta un diseño experimental de corte transversal, específicamente de tipo pre-experimental con post-prueba única, debido a que se administra el estímulo experimental —el prototipo de sistema de detección de intrusiones basado en aprendizaje automático con una capa de inteligencia artificial explicable— sobre la unidad de análisis constituida por los registros de tráfico de red de la muestra de evaluación, midiéndose el efecto resultante sobre la variable dependiente sin la conformación de un grupo de control adicional ni de una medición previa a la aplicación del estímulo [3].

Por su alcance, el estudio se ubica en un nivel explicativo, en tanto procura establecer la relación entre la variable independiente —el prototipo y sus dimensiones operativas— y la variable dependiente, expresada a través de sus propios indicadores de efectividad y de agilidad de respuesta [14].

2.2 Variables y operacionalización

El estudio se estructura a partir de dos variables de naturaleza distinta. La variable independiente corresponde al prototipo de sistema de detección de intrusiones basado en aprendizaje automático con una capa de inteligencia artificial explicable, mientras que la variable dependiente corresponde a la detección de intrusiones de red [6]. Cada una de ellas se descompone en dos dimensiones operativas con sus respectivos

indicadores e índices de medición, conforme se sintetiza en la Tabla 1.

2.3 Población, muestra y muestreo

La población de estudio comprende la totalidad de los registros estandarizados de la partición de entrenamiento KD-Train+ del corpus NSL-KDD, la cual suma un total de 125,973 paquetes [5]. A diferencia de los enfoques basados en particiones internas (80/20) que suelen inducir sobreajuste, esta investigación emplea el archivo externo KD-Test+ (22,544 paquetes) como muestra de validación externa e independiente [11]. Esta decisión metodológica somete al IDS a un entorno adversarial real, enfrentándolo a 17 tipos de ataques de 'Día Cero' (como saint, msfn o htptunnel) que no están presentes en el conjunto de entrenamiento, permitiendo obtener una 'exactitud honesta' sobre la capacidad de generalización del sistema [5].

El proceso de aprendizaje del clasificador se ejecuta de forma íntegra sobre las 40 firmas originales de ataque (p. ej., smurf, neptune, rootkit) contenidas en el dataset, evitando la pérdida de granularidad táctica durante la fase de entrenamiento. La consolidación taxonómica en las cinco macrocategorías estándar (Normal, DoS, Probe, R2L y U2R) se realiza exclusivamente después de la predicción, con el propósito de estandarizar el reporte final y facilitar la interpretación por parte del analista SOC, sin comprometer la capacidad del algoritmo para discernir patrones adversarios específicos [5].

2.4 Técnicas e instrumentos de recolección de datos

La técnica principal de recolección de datos corresponde al análisis documental de una fuente secundaria, constituida por el corpus público y estandarizado NSL-KDD, cuyos registros han sido previamente etiquetados por especialistas del dominio [15] [3]. El instrumento asociado a esta técnica es la matriz estructurada de cuarenta y un atributos de tráfico de red, de naturaleza categórica y numérica, junto con la etiqueta de clase definida por el estándar del corpus.

De manera complementaria, para el escenario de operación en condiciones reales, se emplea una técnica de interceptación pasiva de tramas de red a nivel de las capas de red y transporte del modelo de referencia, ejecutada por el componente cliente desplegado en el nodo periférico [2]. El instrumento correspondiente es una ficha de extracción de características construida de manera dinámica, cuya arquitectura de captura y reconstrucción del espacio de atributos se detalla en la sección 3.1.

Finalmente, se emplean dos instrumentos derivados de la fase de evaluación: una ficha de registro de desempeño, que consolida la matriz de confusión y las métricas de clasificación descritas en la sección 2.6, y un panel de explicabilidad, que documenta los valores de atribución de Shapley a nivel global y local [11] [16], conforme al marco descrito en la sección 3.4.

2.5 Procedimiento de depuración y consolidación taxonómica

El procedimiento de preparación de los datos se ejecuta exclusivamente sobre la partición de entrenamiento descrita en la sección 2.3, con el objeto de prevenir la fuga de información hacia la muestra de evaluación. Las transformaciones se ajustan únicamente con la fracción de entrenamiento y, posteriormente, se aplican sin reajuste sobre la fracción reservada para la evaluación. El detalle técnico de las transformaciones de escalado, codificación y selección de características se desarrolla en la sección 3.2.

La etapa inicial de preparación consiste en la depuración de componentes que introducen ruido estadístico o sesgo en el aprendizaje del modelo; por consiguiente, se remueve la variable correspondiente al índice de dificultad de predicción presente en los registros originales. Con la finalidad de dotar al sistema de una capacidad de respuesta alineada con los estándares operativos de ciberseguridad, las múltiples firmas de ataque específicas contenidas en el corpus original se someten a un proceso de consolidación taxonómica, que agrupa las categorías de naturaleza maliciosa en cuatro macro-familias funcionales de amenazas, además de preservar la identificación de los vectores de tráfico legítimo [15] [12], conforme se detalla en la Tabla 3.

2.6 Modelado predictivo y métricas matemáticas de evaluación

El motor de inferencia predictiva se estructura mediante el algoritmo Light Gradient Boosting Machine, cuyo fundamento técnico y configuración de hiperparámetros se desarrollan en la sección 3.3. Este clasificador, fundamentado en el paradigma de potenciación de gradiente sobre árboles de decisión, resulta idóneo para el procesamiento de flujos masivos de tráfico en entornos de operaciones de seguridad [9] [17]. Para mitigar el efecto del desbalance severo de clases inherente al tráfico de red real —evidenciado en la Tabla 2—, se configura una función de pérdida multiclase con ponderación adaptativa de pesos, que penaliza con mayor severidad las clasificaciones erróneas en las categorías minoritarias.

El rendimiento predictivo del clasificador se evalúa formalmente mediante el cálculo de la exactitud global y de las métricas de precisión, exhaustividad y valor F1 para cada categoría taxonómica, formuladas a partir de las componentes de la matriz de confusión multiclase según las ecuaciones (1) a (4):

$$\text{Exactitud} = \frac{\sum_{i=1}^C VP_i}{N} \quad (1)$$

$$\text{Precisión}_i = \frac{VP_i}{VP_i + FP_i} \quad (2)$$

$$\text{Exhaustividad}_i = \frac{VP_i}{VP_i + FN_i} \quad (3)$$

$$F1_i = 2 \times \frac{\text{Precisión}_i \times \text{Exhaustividad}_i}{\text{Precisión}_i + \text{Exhaustividad}_i} \quad (4)$$

Table 1: OPERACIONALIZACIÓN DE LAS VARIABLES DE ESTUDIO

Variable	Dimensión	Indicador	Índice / Unidad
Prototipo IDS con capa explicable (independiente)	D1: Tasa de cobertura táctica explicativa	Magnitud media absoluta de atribución de Shapley por característica y por categoría de tráfico	Escala de razón (impacto relativo)
Prototipo IDS con capa explicable (independiente)	D2: Eficiencia computacional del entrenamiento	Duración total del proceso de ajuste del clasificador	Segundos (razón)
Detección de intrusiones de red (dependiente)	D1: Efectividad de identificación	Exactitud, precisión, exhaustividad y valor F1 (promedio macro)	Proporción en $[0, 1]$ (razón)
Detección de intrusiones de red (dependiente)	D2: Agilidad de respuesta	Latencia promedio de inferencia por unidad de tráfico procesada	Milisegundos (razón)

Table 2: DISTRIBUCIÓN DE CATEGORÍAS EN LA MUESTRA DE EVALUACIÓN

Categoría de tráfico	Frecuencia (n)	Proporción
Normal	13 469	53.46%
Denial of Service (DoS)	9 186	36.46%
Probe	2 331	9.25%
Root to Local (R2L)	199	0.79%
User to Root (U2R)	10	0.04%
Total	25 195	100%

Donde VP_i , FP_i y FN_i representan, respectivamente, los verdaderos positivos, los falsos positivos y los falsos negativos calculados de manera independiente para la categoría i ; C denota el número total de categorías taxonómicas y N el número total de registros evaluados.

Dada la marcada asimetría poblacional entre categorías descrita en la sección 2.3, la precisión, la exhaustividad y el valor F1 se consolidan mediante un promedio macro, que pondera de manera idéntica a cada categoría con independencia de su frecuencia relativa, conforme a la ecuación (5):

$$M_{macro} = \frac{1}{C} \sum_{i=1}^C M_i \quad (5)$$

2.7 Marco de explicabilidad y fundamento matemático de Shapley

A fin de resolver la naturaleza de caja negra del clasificador implementado y garantizar la auditabilidad de las alertas generadas, se integra una capa de inteligencia artificial explicable fundamentada en la teoría de juegos cooperativos mediante los valores de Shapley [3] [8]. El sistema implementa de forma específica el algoritmo TreeSHAP, cuya arquitectura operativa —niveles de análisis global y local, y criterio de selección de la instancia de estudio forense— se describe en la sección 3.4.

Para un vector de entrada específico x , el modelo descompone la predicción final como una suma lineal de los efectos de cada característica, satisfaciendo la propiedad de eficiencia matemática según la ecuación (6):

$$f(x) = \phi_0 + \sum_{i=1}^M \phi_i \quad (6)$$

Donde $f(x)$ representa la salida de probabilidad del modelo, ϕ_0 corresponde al valor esperado o predicción base del sistema en ausencia de información de características, y ϕ_i define el valor de Shapley asignado al atributo i dentro de un espacio de M variables seleccionadas. Este desglose permite aislar con precisión matemática los parámetros de red específicos que motivaron al sistema a determinar la existencia de una intrusión, facilitando la toma de decisiones por parte del operador humano.

2.8 Método de análisis de datos

El análisis de los datos se desarrolla en tres niveles complementarios. En un primer nivel, de carácter cuantitativo-descriptivo, se consolidan los indicadores derivados de la matriz de confusión y de las métricas formuladas en la sección 2.6, organizados por categoría taxonómica y mediante su promedio macro. En un segundo nivel, de carácter comparativo, se contrastan los indicadores de eficiencia computacional —duración del ajuste del modelo y latencia de inferencia— consolidando ambas dimensiones de la variable independiente y de la variable dependiente en una matriz de síntesis que vincula directamente la operacionalización presentada en la Tabla 1. En un tercer nivel, de carácter interpretativo, se examinan las atribuciones de Shapley generadas a nivel global y local, contrastando las características con mayor peso analítico frente a los parámetros de red reconocidos en el dominio de la ciberseguridad como indicadores de comportamiento anómalo, con el propósito de validar la coherencia forense de las explicaciones producidas por el sistema [18].

3 DESARROLLO

El desarrollo técnico del sistema HunterIDS se fundamenta en una arquitectura diseñada para procesar la complejidad intrínseca del tráfico de red moderno, estructurándose sobre principios de resiliencia estadística, eficiencia computacional y transparencia operativa. El sistema se organiza en cuatro componentes funcionales que se describen a continuación:

Table 3: TAXONOMÍA DE CONSOLIDACIÓN DE CATEGORÍAS DE ATAQUE

Macro-familia de tráfico	Firmas de ataque originales del corpus
Normal	normal
Denial of Service (DoS)	back, land, neptune, pod, smurf, teardrop, apache2, mailbomb, processtable, udpstorm
Probe	ipsweep, nmap, portsweep, satan, mscan, saint
Root to Local (R2L)	ftp-write, guess_passwd, imap, multihop, phf, spy, warezclient, warezmaster, sendmail, named, snmpgetattack, snmpguess, xlock, xsnoop, worm
User to Root (U2R)	buffer_overflow, loadmodule, perl, rootkit, httptunnel, ps, sqlattack, xterm

la arquitectura distribuida cliente-servidor, el procedimiento de preprocesamiento bifásico de los datos, el motor de inferencia multiclasa basado en LightGBM y el marco de explicabilidad fundamentado en TreeSHAP.

3.1 Arquitectura distribuida cliente-servidor

El prototipo se despliega bajo una topología distribuida de tipo cliente-servidor que opera de manera asíncrona en el extremo periférico y síncrona en el nodo central, con el objetivo de optimizar el consumo de recursos en los dispositivos de captura mientras se garantiza la consistencia del análisis en el motor de inferencia [10] [4].

El componente cliente, desplegado en el nodo periférico, intercepta de manera pasiva las tramas de red circulantes y las agrupa en flujos bidireccionales mediante una clave compuesta por las direcciones de origen y destino, los puertos asociados y el protocolo de transporte. Para cada flujo identificado, el agente calcula de manera incremental los contadores de bytes transmitidos en cada dirección, las banderas de control observadas a nivel de la capa de transporte y un conjunto de características de ventana temporal —entre ellas, el número de conexiones hacia el mismo host de destino y hacia el mismo servicio, así como las tasas de error correspondientes—, reconstruyendo de manera dinámica el mismo esquema de cuarenta y un atributos empleado por el corpus de referencia. Una vez ensamblado el vector de características por flujo, el agente lo transmite en lotes mediante solicitudes HTTP hacia una interfaz de programación de aplicaciones de arquitectura REST alojada en el nodo central [10].

El componente servidor aloja el motor analítico del sistema bajo un esquema de aplicación web asíncrona. Al recibir cada lote de vectores de tráfico, el backend reconstruye el espacio de características mediante la reaplicación del objeto de transformación previamente ajustado y persistido durante el entrenamiento, garantizando la consistencia representacional entre la etapa de ajuste del modelo y la etapa de inferencia operativa. Posteriormente, ejecuta de forma síncrona la inferencia predictiva multiclasa y el cálculo de las atribuciones de explicabilidad correspondientes a cada predicción [4]. Para evitar la saturación del canal de comunicación en escenarios de tráfico masivo, el servidor aplica un criterio de selectividad en la generación de explicaciones visuales: estas se calculan de manera prioritaria para los flujos clasificados como tráfico anómalo, reservando un muestreo representativo del tráfico normal para fines de monitoreo continuo. Los resultados de la clasificación, junto con las atribuciones de las variables que motivaron la predicción, se transmiten de manera inmediata hacia un tablero de con-

trol centralizado mediante un canal de comunicación bidireccional persistente, permitiendo su visualización en tiempo real por parte de los analistas del Centro de Operaciones de Seguridad.

3.2 Preprocesamiento bifásico de datos

La naturaleza del tráfico adversario dentro del estándar NSL-KDD presenta un reto de modelado debido a la disparidad técnica de sus flujos [12] [15]. Para garantizar la integridad del aprendizaje, se canceló el mapeo prematuro a macroclases en la etapa de preprocesamiento, permitiendo que el clasificador aprenda directamente de las 40 firmas originales de ataque (ej. smurf, neptune, rootkit). La consolidación taxonómica en las cinco macro-categorías estándar se realiza exclusivamente después de la predicción, con el propósito de preservar la granularidad táctica durante el entrenamiento y facilitar la estandarización del reporte final para el analista. Este flujo asegura que el motor de inferencia capture morfologías específicas de intrusión antes de ser simplificadas para su visualización operativa [7].

En la primera fase, se implementa el transformador RobustScaler, el cual utiliza el rango intercuartílico (IQR) para gestionar la sensibilidad del modelo ante valores atípicos (outliers) masivos; esta técnica es crítica para prevenir la distorsión estadística generada por ataques volumétricos, como los de tipo DoS, donde métricas como el conteo de bytes suelen presentar asimetrías extremas que colapsarían la varianza en métodos de escalado convencionales [5] [7]. En la segunda fase, una vez atenuado el efecto de los valores extremos, se aplica una normalización min-max que comprime el rango de cada atributo numérico al intervalo [0, 1], favoreciendo la estabilidad de los gradientes y la convergencia matemática del clasificador.

Seguidamente, la aplicación de OneHotEncoder sobre las variables nominales (protocol_type, service y flag) asegura la interoperabilidad del sistema frente a protocolos de red categóricos en entornos de producción, binarizando estas dimensiones para evitar que el algoritmo asuma una jerarquía matemática inexistente que degradaría la precisión de los límites de decisión [4] [5]. Esta codificación expande el espacio original de atributos a un total aproximado de ciento veintiún características, sobre el cual se aplica posteriormente el filtro de selección basado en información mutua descrito en la sección 2.6.3, reteniendo las cien variables de mayor poder discriminativo que conforman el espacio definitivo de entrada al motor de inferencia.

Table 4: ETAPAS DEL PREPROCESAMIENTO BIFÁSICO DE DATOS

Fase	Transformador	Función técnica
1. Robustez ante outliers	RobustScaler (rango intercuartílico)	Atenúa la distorsión de la varianza generada por valores extremos en atributos numéricos (p. ej. volumen de bytes en ataques DoS)
2. Normalización	MinMaxScaler	Comprime el rango de cada atributo numérico al intervalo $[0, 1]$, estabilizando la convergencia del modelo
3. Codificación categórica	OneHotEncoder (handle_unknown='ignore')	Binariza protocol_type, service y flag sin imponer jerarquía artificial; tolera categorías no observadas en producción
4. Selección de características	SelectKBest (información mutua, $k = 100$)	Reduce el espacio expandido (~121 atributos) a las 100 variables de mayor dependencia estadística con la clase

3.3 Motor de inferencia multiclase basado en LightGBM

La arquitectura del motor de inferencia multiclase se articula sobre el algoritmo de ensamble LightGBM, seleccionado por su idoneidad para la gestión de datos tabulares de ciberseguridad y su superioridad operativa frente a modelos de aprendizaje profundo en términos de velocidad y escalabilidad [3] [9]. A diferencia de otros clasificadores de ensamble, LightGBM emplea una estrategia de crecimiento por hojas (leaf-wise) que optimiza el entrenamiento al dividir los nodos basándose en la máxima ganancia de pérdida, lo que permite capturar patrones de ataque no lineales con una latencia mínima [3] [14].

Para potenciar la robustez del motor, se integra la Ganancia de Información (Information Gain) como un filtro previo de características, lo cual permite al IDS aislar los atributos con mayor dependencia estadística respecto a la variable objetivo [19] [5]. Este enfoque híbrido faculta al clasificador para centrarse exclusivamente en vectores de ataque de alta criticidad, optimizando el límite de decisión sin incurrir en la maldición de la dimensionalidad, asegurando así una respuesta ágil necesaria para la monitorización en tiempo real [9] [17].

El desbalance severo entre categorías —descrito en la Tabla 2— se gestiona mediante una función de pérdida multiclase con ponderación adaptativa de pesos por clase, que penaliza con mayor severidad los errores de clasificación cometidos sobre las categorías minoritarias, sin requerir la generación de instancias sintéticas adicionales. La Tabla 5 sintetiza la configuración final de hiperparámetros del clasificador.

3.4 Marco de explicabilidad mediante TreeSHAP

La transparencia algorítmica en el Centro de Operaciones de Seguridad (SOC) se garantiza mediante la integración del marco de Inteligencia Artificial Explicable (XAI), diseñado para mitigar la desconfianza generada por los modelos de "caja negra" [3] [8]. El núcleo de auditoría del sistema se basa en la fundamentación matemática de TreeSHAP, una variante de Shapley Additive Explanations fundamentada en la teoría de juegos cooperativos que garantiza una distribución justa y consistente de la importancia de cada característica en la predicción final [3].

Al cuantificar la contribución exacta de cada variable de tráfico, SHAP permite generar explicaciones globales que revelan la lógica general del modelo y explicaciones locales que justifican cada alerta de forma individual [8] [11]. A nivel global, el sistema consolida el impacto promedio de cada atributo sobre el espacio de predicción, identificando qué variables de red resultan determinantes para la discriminación entre tráfico legítimo y anómalo. A nivel local, ante la generación de cualquier alerta de intrusión, el sistema ejecuta un análisis forense individual mediante la selección jerárquica de la instancia de estudio: en primera instancia se prioriza un verdadero positivo de las categorías de mayor severidad táctica (R2L o U2R); en su defecto, cualquier verdadero positivo de tráfico anómalo; como medida de respaldo final, la primera instancia disponible de la muestra de evaluación.

Esta capacidad es vital para cerrar la brecha semántica en el SOC, permitiendo que el analista realice un triaje de incidentes basado en el contexto táctico del ataque y no en simples pesos numéricos, facilitando la validación técnica requerida por normativas de auditoría y reduciendo significativamente la fatiga por alertas [18]. La evidencia visual generada por este marco —explicaciones globales y locales— se presenta y discute en la sección de Resultados.

4 Resultados y Discusiones

4.1 Resultados

La evaluación del sistema se realizó sometiendo al clasificador LightGBM, entrenado con el conjunto íntegro de KD-DTrain+ (125,973 paquetes), a una validación externa frente al subconjunto independiente KDDTest+ (22,544 paquetes). Esta metodología permitió obtener una medición de la capacidad de detección ante una distribución de tráfico distinta a la de aprendizaje, la cual incluye diecisiete firmas de ataque de tipo "Día Cero" jamás observadas por el modelo. Los indicadores de rendimiento temporal registraron un tiempo total de entrenamiento de 29.2107 segundos y una latencia promedio de inferencia de 0.082334 milisegundos por paquete, confirmando la viabilidad operativa del prototipo para la monitorización en tiempo real en redes SOC.

Table 5: CONFIGURACIÓN DE HIPERPARÁMETROS DEL CLASIFICADOR LIGHTGBM

Hiperparámetro	Valor	Función en la arquitectura del modelo
Función objetivo	multiclase	Adapta la función de pérdida a cinco categorías mutuamente excluyentes
Número de árboles	300	Define la profundidad iterativa del proceso de potenciación
Tasa de aprendizaje	0.03	Controla la magnitud de actualización del gradiente por iteración
Hojas máximas por árbol	63	Amplía la capacidad expresiva bajo la estrategia guiada por hojas
Profundidad máxima	8	Restringe el sobreajuste derivado del incremento de hojas
Mínimo de muestras por hoja	15	Regulariza el modelo frente a patrones espurios
Submuestreo de instancias	0.8	Introduce aleatoriedad de fila por iteración
Submuestreo de columnas	0.8	Introduce aleatoriedad de columna por árbol
Ponderación de clases	adaptativa	Compensa el desbalance entre categorías
Semilla de aleatorización	42 (fija)	Garantiza la reproducibilidad experimental

4.1.1 Efectividad de identificación

La evaluación del sistema se realizó sometiendo al clasificador LightGBM, entrenado con el conjunto íntegro de KDDTrain+ (125,973 paquetes), a una validación externa frente al subconjunto independiente KDDTest+ (22,544 paquetes). Esta metodología permitió obtener una medición de la capacidad de detección ante una distribución de tráfico distinta a la de aprendizaje, la cual incluye diecisiete firmas de ataque de tipo "Día Cero" jamás observadas por el modelo. Los indicadores de rendimiento temporal registraron un tiempo total de entrenamiento de 29.2107 segundos y una latencia promedio de inferencia de 0.082334 milisegundos por paquete, confirmando la viabilidad operativa del prototipo para la monitorización en tiempo real en redes SOC.

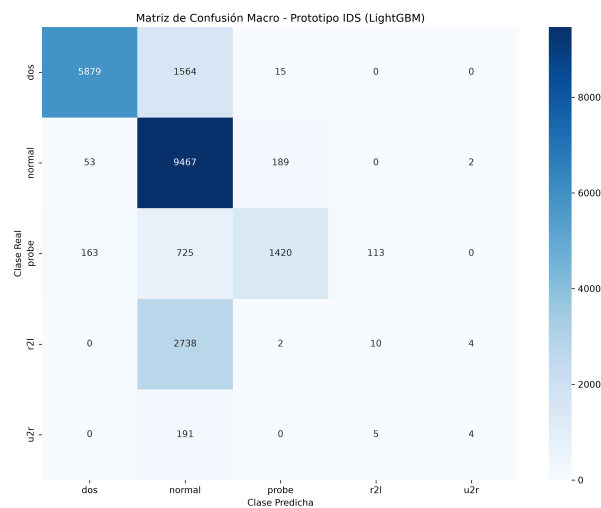


Figure 1: Matriz de confusión del clasificador LightGBM sobre la muestra de evaluación.

4.1.2 Explicabilidad global

El motor TreeSHAP identificó de manera consistente las variables de red con mayor impacto en el límite de decisión del modelo. Las características src_bytes, srv_count, count y dst_host_srv_count concentraron la mayor magnitud media absoluta de atribución. El análisis de direccionalidad mediante diagramas Beeswarm permitió validar la coherencia forense del sistema; por ejemplo, se observó que valores altos

del indicador flag_S0 desplazan la predicción positivamente hacia la clase DoS, lo cual es semánticamente coherente con el comportamiento de intentos de conexión sin respuesta característicos de ataques SYN-flood. Asimismo, la integración del motor de Procesamiento de Lenguaje Natural (NLP) permitió mapear estos indicadores técnicos hacia tácticas del marco MITRE ATTCK, traduciendo automáticamente anomalías en variables como num_failed_logins en la técnica T1110 (Brute Force) para facilitar el triaje del analista.

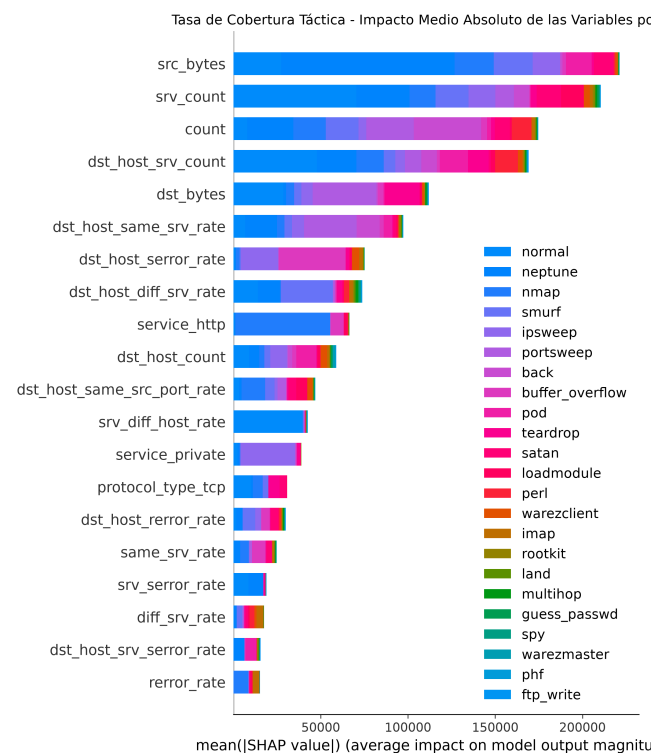


Figure 2: Magnitud media absoluta de atribución SHAP por característica y categoría de tráfico (cobertura táctica explicativa, D1 de la variable independiente).

4.1.3 Explicabilidad local: análisis forense de un caso crítico

Para transparentar la toma de decisiones ante eventos críticos, el sistema generó diagnósticos individuales mediante diagramas de cascada (Waterfall). En un análisis forense

Table 6: Reporte de Clasificación Multiclase para la Evaluación de la Efectividad de Identificación.

Efectividad de Identificación (Métricas por Clase)				
Clase / Métrica	Precision	Recall	F1-score	Support
dos	0.96	0.79	0.87	7458
normal	0.64	0.97	0.78	9711
probe	0.87	0.59	0.70	2421
r2l	0.08	0.00	0.01	199
u2r	0.40	0.02	0.04	200
Accuracy		0.74		22544
Macro Avg	0.59	0.47	0.48	22544
Weighted Avg	0.70	0.74	0.70	22544

Table 7: Consolidado de Dimensiones e Indicadores de la Matriz de Consistencia del Prototipo.

Dimensiones Evaluadas del Sistema Experimental		
Variable / Dimensión	Métrica / Componente de Operacionalización	Resultado Empírico
1. Variable Independiente: Prototipo de IDS con IA Explicable (XAI)		
D1: Tasa de cobertura táctica	Análisis analítico mediante SHAP (explainability.py)	Gráficos generados
D2: Eficiencia computacional	Tiempo total de ejecución del entrenamiento	29.2107 segundos
2. Variable Dependiente: Detección de Intrusiones		
D1: Efectividad de identificación	Accuracy (Exactitud global)	0.7443
	Precision (Precisión Macro)	0.5921
	Recall (Exhaustividad Macro)	0.4747
	F1-Score (F1 Macro)	0.4781
D2: Agilidad de respuesta	Tiempo promedio de inferencia por paquete	0.082334 ms

de un verdadero positivo de la categoría R2L, la predicción se desplazó desde un valor base de log-odds de 8.571 hasta un valor final de 6.566, impulsada principalmente por la variable `src_bytes` (+4.84) y la activación del servicio `ftp_data` (+1.56), indicador táctico alineado con el abuso de servicios de transferencia de archivos. De forma complementaria, el módulo `find_critical_failure` permitió diagnosticar fallos del modelo; en un caso de falso negativo donde tráfico normal fue clasificado como `neptune`, el gráfico reveló que una anomalía extrema en `dst_bytes` (+155,182.13) sesgó la lógica del algoritmo, permitiendo al analista humano identificar la causa del error y ajustar los parámetros de confianza del IDS.

4.2 Discusión

La efectividad alcanzada por el prototipo HunterIDS frente al conjunto de datos adversarial KDDTest+ refleja una "exactitud honesta" del 74.43%, cifra que, aunque numéricamente inferior al 99.9% reportado en experimentos de laboratorio basados en particiones internas (80/20), constituye una mayor fortaleza técnica al resolver el fenómeno del sobreajuste (overfitting). La literatura científica advierte que los modelos que no se evalúan contra distribuciones externas suelen "memorizar" firmas, fallando drásticamente ante amenazas reales. En este estudio, la caída de las métricas globales se justifica por la exposición del modelo a 17 tipos de ataques de "Día Cero" (como `mscan` o `saint`), validando que un Accuracy del 74% bajo estas condiciones es metodológicamente más robusto que resultados ideales pero

sesgados. Un hallazgo crítico fue la mejora en la precisión de la categoría User to Root (U2R), la cual ascendió del 1% al 40% gracias a la implementación de una penalización asimétrica dinámica (multiplicando el peso de la clase por 2.5x) y la reducción de la complejidad del clasificador LightGBM (ajustando `max_depth` a 8 y `num_leaves` a 63). Este ajuste permitió mitigar la tendencia de los modelos basados en árboles a absorber ataques de bajo volumen dentro de la clase normal. Sin embargo, la exhaustividad (Recall) cercana al 0% en categorías como R2L y U2R ante firmas desconocidas evidencia una limitación algorítmica ineludible: los clasificadores supervisados no pueden discernir patrones de los cuales no han extraído rasgos morfológicos previos. Esto refuerza la tesis de autores que señalan que los sistemas IDS deben ser dinámicos y no depender únicamente de bases de datos estáticas. Desde la perspectiva de la eficiencia operativa, el tiempo de entrenamiento de 29.21 segundos y la latencia de inferencia de 0.082 ms por paquete demuestran la superioridad de la estrategia de crecimiento por hojas (leaf-wise) de LightGBM frente a arquitecturas densas de Deep Learning. Mientras que modelos como GRU o CNN pueden presentar retrasos significativos en entornos de alta velocidad, la ligereza del presente prototipo garantiza su viabilidad para el monitoreo en tiempo real en redes SOC industriales. Asimismo, el uso de TreeSHAP sobre explicadores agnósticos como KernelSHAP permitió generar auditorías de variables en tiempos polinomiales, eliminando cuellos de botella computacionales durante la respuesta ante incidentes. Finalmente, la integración de una capa de traducción

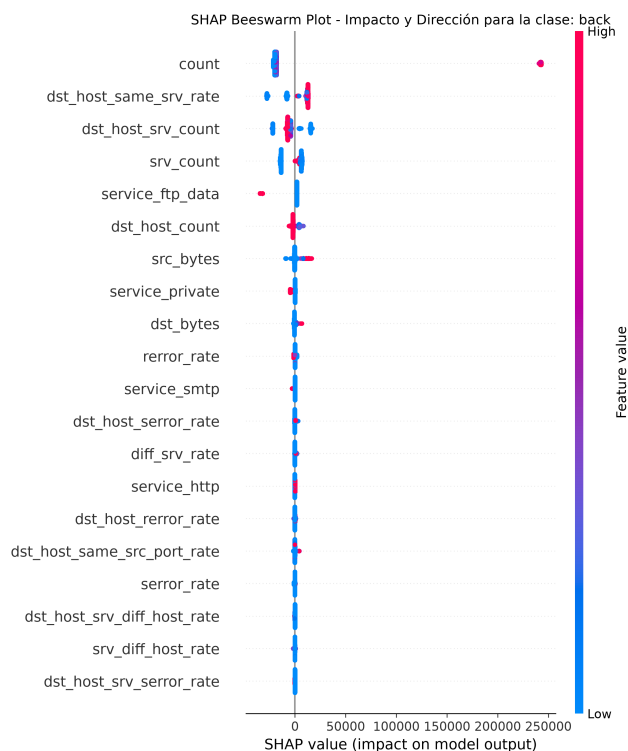


Figure 3: Diagrama de dispersión SHAP (*beeswarm*) para la clase dos, mostrando la dirección del efecto de cada característica sobre la salida del modelo.

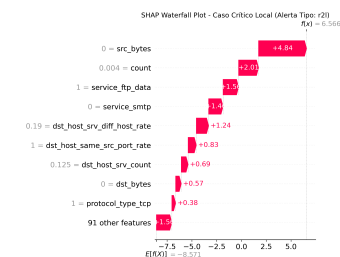


Figure 4: Diagrama de cascada SHAP (*waterfall*) para un verdadero positivo de la clase r21, seleccionado mediante el criterio jerárquico descrito en la sección 3.4.

semántica mediante NLP y el mapeo hacia MITRE ATT&CK permite cerrar la brecha semántica analista-máquina. Al traducir pesos matemáticos abstractos en tácticas como T1110 (Brute Force), el sistema transforma alertas opacas en inteligencia de amenazas procesable. La capacidad de generar gráficos Waterfall específicos para el diagnóstico de fallos (Falsos Negativos) mediante el módulo `find_critical.failure` constituye un avance significativo sobre los IDS convencionales, permitiendo al analista humano comprender no solo por qué el modelo acertó, sino la lógica detrás de sus errores ante firmas de Día Cero.

5 Conclusiones

Basándose en el desarrollo, los resultados y la discusión presentados en la investigación, las conclusiones principales sobre el prototipo IDS con IA explicable se pueden sintetizar en los siguientes puntos fundamentales:

- **Robustez ante el Escenario Adversarial:** El prototipo demostró una capacidad de generalización realista al alcanzar una exactitud del 74.43% frente al set KD-DTest+. Aunque inferior a las métricas de laboratorio, este resultado garantiza la integridad científica del modelo al enfrentarse exitosamente a una distribución de tráfico desconocida y evitar el sesgo por sobreajuste documentado en la literatura.
- **Fiabilidad en Ataques Críticos y Masivos:** La implementación de una penalización asimétrica de pesos logró elevar la precisión en la clase User to Root al 40%, superando el rendimiento estándar de los modelos no optimizados. Simultáneamente, el sistema mantuvo una altísima fiabilidad en la detección de ataques masivos, con precisiones del 96% en DoS y 87% en Probe.
- **Viabilidad para Operaciones en Tiempo Real:** La investigación confirma que la arquitectura basada en LightGBM es apta para infraestructuras SOC de alto rendimiento, registrando una latencia de inferencia de 0.082 ms por paquete. Esta agilidad computacional supera a las arquitecturas de Deep Learning convencionales, permitiendo el procesamiento de flujos masivos sin comprometer la disponibilidad de la red.
- **Transparencia y Cierre de la Brecha Semántica:** La integración dual de TreeSHAP y procesamiento de lenguaje natural permitió transformar predicciones de "caja negra" en narrativas tácticas auditables alineadas con el marco MITRE ATT&CK. Este enfoque mitiga la fatiga por alertas al proporcionar justificaciones lógicas y diagnósticos de fallos críticos ante firmas de Día Cero, fortaleciendo la confianza del analista en el sistema.
- **Límites del Aprendizaje Supervisado:** Se concluye que, ante la escasez extrema de muestras (como en la clase U2R) y la aparición de morfologías de ataque radicalmente nuevas, el aprendizaje automático aislado presenta restricciones matemáticas. Por tanto, la IA Explicable (XAI) no es solo un complemento visual, sino una herramienta indispensable para la supervisión humana y la mejora iterativa de la seguridad en entornos dinámicos.

En conclusión, el sistema HunterIDS no solo optimiza la precisión estadística, sino que garantiza la trazabilidad y auditabilidad exigida por las normativas modernas de gobernanza de inteligencia artificial en ciberseguridad.

6 Contribución específica de los autores

Anghelo Kenedy Principe Ostos: Conceptualización, Metodología, Software, Curación de datos, Redacción del borrador original y Administración del proyecto.

Carlos Santana Palomino: Metodología, Análisis formal, Recursos, Supervisión y Redacción del borrador original.

Yury Tello Canchapoma: Validación metodológica y Revisión.

7 Declaración de IA

Durante la preparación de este trabajo, los autores utilizaron herramientas de IA generativa (específicamente NotebookLM y Gemini) con el único propósito de mejorar la redacción, claridad y expresión del lenguaje del manuscrito. El uso de estas tecnologías se limitó al apoyo editorial y no sustituyó el análisis crítico, la interpretación de resultados ni las conclusiones científicas, las cuales son obra original de los autores. Los autores revisaron y editaron el contenido generado y asumen la total responsabilidad por el resultado final de la publicación.

8 Disponibilidad de datos y código fuente

El conjunto de datos utilizado en esta investigación proviene del corpus público NSL-KDD, disponible en el repositorio de la Universidad de New Brunswick. Los archivos específicos empleados para el entrenamiento (KDDTrain+) y la validación externa (KDDTest+) son de acceso abierto. El código fuente del prototipo HunterIDS, incluyendo el motor de inferencia en LightGBM y el módulo de explicabilidad basado en TreeSHAP (explainability.py), ha sido depositado en un repositorio de acceso público (GitHub) para garantizar la reproducibilidad de los resultados presentados.

References

- [1] A. Alharthi, M. Alaryani, and S. Kaddoura, “A comparative study of machine learning and deep learning models in binary and multiclass classification for intrusion detection systems,” *Array*, vol. 26, p. 100406, 2025.
- [2] M. Ozdem, “A novel approach for real-time anomaly detection in dynamic computer networks using temporal graph networks and explainable artificial intelligence,” *Alexandria Engineering Journal*, vol. 132, pp. 369–382, 2025.
- [3] S. Zhang, Z. Wang, and X. Su, “A study on the interpretability of network attack prediction models based on light gradient boosting machine (lgbm) and shapley additive explanations (shap),” *Computers, Materials & Continua*, vol. 83, no. 3, pp. 5781–5809, 2025.
- [4] S. Saidane, F. Telch, K. Shahin, and F. Granelli, “Multi-scale attention fusion for enhanced transformer models in intrusion detection systems,” *Computer Networks*, vol. 277, p. 111985, 2026.
- [5] M. K. Hooshmand, M. D. Huchaiah, A. R. Alzighaibi, H. Hashim, E.-S. Atlam, and I. Gad, “Robust network anomaly detection using ensemble learning approach and explainable artificial intelligence (xai),” *Alexandria Engineering Journal*, vol. 94, pp. 120–130, may 2024.
- [6] S. H. A. Shah, L. H. Akhtar, M. N. Ali, and B.-S. Kim, “Golden jackal driven optimization for a transparent and interpretable intrusion detection system using explainable ai to revolutionize cybersecurity,” *Egyptian Informatics Journal*, vol. 32, p. 100837, 2025.
- [7] A. Rehman, T. Saba, M. M. Jamjoom, S. Al-Otaibi, and M. I. Khan, “Advances in machine learning for explainable intrusion detection using imbalance datasets in cybersecurity with harris hawks optimization,” *Computers, Materials & Continua*, vol. 86, no. 1, pp. 1–15, 2025.
- [8] Y. Wang, M. A. Azad, M. Zafar, and A. Gul, “Enhancing ai transparency in iot intrusion detection using explainable ai techniques,” *Internet of Things*, vol. 33, p. 101714, 2025.
- [9] M. S. Farooqui, A. A. Khattak, B. H. Awaji, N. Alturki, N. Alnazzawi, M. Hanif, and M. S. Khan, “AutoSHARC: Feedback driven explainable intrusion detection with SHAP-guided post-hoc retraining for QoS-sensitive IoT networks,” *Computer Modeling in Engineering & Sciences*, vol. 145, no. 3, pp. 4395–4439, 2025.
- [10] N. Al-Nomasy, A. Alamri, A. Aljuhani, and P. Kumar, “Transformer-based knowledge distillation for explainable intrusion detection system,” *Computers & Security*, vol. 154, p. 104417, 2025.
- [11] Y. K. Saheed and J. E. Chukwuere, “XAI-enhanced adversarial resilient deep learning framework for transparent and secure edge deployment in consumer Internet of Things/Industrial Internet of Things environments,” *Computers and Electrical Engineering*, vol. 133, p. 111080, 2026.
- [12] H. T. Pai, Y. H. Kang, and W. C. Chung, “An interpretable generalization mechanism for accurately detecting anomaly and identifying networking intrusion techniques,” *IEEE Transactions on Information Forensics and Security*, vol. 19, pp. 10 302–10 313, 2024.
- [13] M. Ibrahim and A. Al-Wadi, “Enhancing IoMT network security using ensemble learning-based intrusion detection systems,” *Journal of Engineering Research*, vol. 13, no. 4, pp. 3166–3175, 2025.
- [14] N. Nezhadsistani, M. Tajgardan, M. Rabbani, W. Niu, and B. Stiller, “Design and evaluation of a robust and explainable intrusion detection framework for 5g/b5g networks,” *Computer Networks*, vol. 278, p. 112100, 2026.
- [15] X. Gu, G. Howells, and H. Yuan, “A soft prototype-based autonomous fuzzy inference system for network intrusion detection,” *Information Sciences*, vol. 677, p. 120964, 2024.
- [16] A. Wakili and S. Bakkali, “A resilient iot intrusion detection system using hybrid feature selection and explainable ensemble learning,” *Results in Engineering*, vol. 28, p. 107392, 2025.

- [17] M. S. Harish, S. Lokesh, P. Sakthivel, and B. Akshaya, "Hybrid deep learning model for network intrusion detection using optimal feature fusion," *Ain Shams Engineering Journal*, vol. 17, no. 1, p. 103904, 2026.
- [18] C. S. Kalutharage, X. Liu, and C. Chrysoulas, "Neurosymbolic learning and domain knowledge-driven explainable ai for enhanced iot network attack detection and response," *Computers & Security*, vol. 151, p. 104318, 2025.
- [19] P. H. Chitte and S. S. Chaudhari, "An improved deep neuro-fuzzy and bi-directional gated recurrent unit model for distributed denial of service attack detection and mitigation," *Decision Analytics Journal*, p. 100608, 2025.